COMP 2011 Midterm Exam - Spring 2026 - HKUST

Date: Mar 27, 2026 (Friday)
Time Allowed: 2 hours, 7:30pm – 9:30pm
Instructions:

1. This is a closed-book, closed-notes examination.
2. There are 8 questions on **30** pages (including this cover page, honor code, and 3 blank pages at the end).
3. Write your answers in the space provided in black/blue ink. *NO pencil please, otherwise you are not allowed to appeal for any grading disagreements.*
4. All programming codes in your answers must be written in the ANSI C++11 version as taught in the class.
5. For programming questions, you are **NOT** allowed to define additional helper functions or structures, nor global variables, unless otherwise stated. You **cannot** use any library functions not mentioned, nor the `auto` keyword in defining identifiers in the questions.

Student Name	
Student ID	
Venue and Seat Number	

Problem	**Score**
1. Basics	/ 5
2. Control Flow	/ 5
3. Reference/Function and Parameter passing	/ 9
4. Scope	/ 10
5. Array/C String	/ 10
6. Recursion	/ 10
7. Function and Structure	/ 25
8. 2D Array	/ 26
Total	/ 100

The HKUST Academic Honor Code

Honesty and integrity are central to the academic work of HKUST. Students of the University must observe and uphold the highest standards of academic integrity and honesty in all the work they do throughout their program of study.
As members of the University community, students have the responsibility to help maintain the academic reputation of HKUST in its academic endeavors.
Sanctions will be imposed if students are found to have violated the regulations governing academic integrity and honesty.

Declaration of Academic Integrity
I confirm that I have answered the questions using only materials specifically approved for use in this examination, that all the answers are my own work, and that I have not received any assistance during the examination.

Signature: _____

Consent for Use of Exam Scripts in AI Training

We are seeking your consent to use your exam scripts for the purpose of training an artificial intelligence system that will enhance our online and onsite computer-based examination platform. This AI will help improve the quality and efficiency of our assessments. Please note that your identity, including your name and student ID, **will not** be used in the training process.

Your contributions are invaluable, and we want to ensure that you are comfortable with this process. If you agree to allow us to use your exam scripts for this purpose, please sign to indicate your consent. Thank you for your support in helping us create a better examination experience for everyone.

I confirm that I agree to allow my exam scripts to be used for training the AI.

Signature: _____
You are not required to sign if you do not agree to allow your exam scripts to be used.

Problem 1 [5 points] Basics

Operator	Description	Associativity
`()`	parentheses	—
`++ -- ! -`	increment, decrement, logical NOT, unary minus	Right-to-Left
`* / %`	multiply, divide, mod	Left-to-Right
`+ -`	add, subtract	Left-to-Right
`> >= < <=`	relational operator	Left-to-Right
`== !=`	equal, not equal	Left-to-Right
`&&`	logical AND	Left-to-Right
`||`	logical OR	Left-to-Right
`=`	assignment	Right-to-Left

Operators are shown in descending order of precedence in the above table. What is the output of the program?

```
#include <iostream>
using namespace std;

int main() {
    int x = 4;
    int y = 7;
    bool z = true;

    cout << (x++ > 4 || y == 7) << endl;

    cout << (--y <= 5 && ++x == 6) << endl;

    cout << (!z || x >= 5 && y == 6) << endl;

    cout << ( (x < y) ? ++x : y-- ) << endl;

    cout << x + y % 4 << endl;

    return 0;
}
```

Answer:

Problem 2 [5 points] Control Flow

What is the output of the following program when `n = 4`?

```
void printPattern(int n) {   // n = 4
    for (int i = 0; i < n; i++) {

        if (i == 0) {
            for (int j = 0; j < n; j++) {
                cout << "X";
            }
            cout << endl;
        }

        for (int j = 0; j < n; j++) {
            if (i == j) {
                cout << "X";
            }
            else if (j < i) {
                cout << "1";
            }
            else {
                cout << "0";
            }
        }

        cout << endl;
    }
}
```

Write the output exactly as printed by the program.

Answer:

Problem 3 [9 points] Function and Parameter Passing

The following program manages animals entering a barn and calculates their required feed units.

```
#include <iostream>
using namespace std;

enum Animal {PIG, SHEEP, HORSE};
const int BARN_CAPACITY = 10;

int& enterBarn(int& count1, int& count2, int& count3) {
    if (count1 + count3 <= BARN_CAPACITY) {
        count1 += count3;
        count2 += count3;
    }
    return count1;
}

int& updateFeedRates(int arr1[], const int arr2[], int i) {
    arr1[i] = arr2[i]/2;
    return arr1[i];
}

int main() {
    int feedRate1[3] = {4, 3, 6}, feedRate2[3]  = {};
    int pigs = 0, sheep = 0, horses = 0, number = 0;
    int count1 = 6, count2 = 2;
    const int count3 = 2;

    enterBarn(enterBarn(number, pigs, count1), pigs, count1);       // Statement 1
    enterBarn(number, sheep, count2);                               // Statement 2
    enterBarn(number, horses, count1 + count2);                     // Statement 3
    enterBarn(number, horses, count3);                              // Statement 4

    int animalCount[] = { pigs, sheep, horses};

    updateFeedRates(feedRate2, animalCount, 0) += feedRate1[0];     // Statement 5

    const int& tmp1 = feedRate1[1];
    updateFeedRates(feedRate2, animalCount, 1) +=tmp1;              // Statement 6

    const int& tmp2 = updateFeedRates(feedRate2, animalCount, 2);
    tmp2 += feedRate1[2];                                           // Statement 7

    int totalFeed = (pigs * feedRate2[PIG]) + (sheep * feedRate2[SHEEP])
                    + (horses * feedRate2[HORSE]);
    cout << "Total feed: " << totalFeed << endl;

```

```
45      return 0;
46  }
```

(a) Please examine whether the seven statements (annotated as Statement 1-7) can be compiled successfully. For each statement, if it can be compiled successfully, write down "Yes". Otherwise, please explain why a compilation error happens.

Answer:

Statement	**Write down "Yes" or explain why a compilation error happens.**
Statement 1	
Statement 2	
Statement 3	
Statement 4	
Statement 5	
Statement 6	
Statement 7	

(b) Assume that the statement(s) with compilation error(s) have been removed from the program, and the remaining program can be compiled successfully. What is the output of the remaining program?

Answer:

```
Total feed:
```

Problem 4 [10 points] Scope

Consider the following program:

```
#include <iostream>
using namespace std;

int a = 10;

void op1(int a) {
    a = a + 5;
    cout << a << endl;
}

void op2(int& a) {
    a = a - 2;
    cout << a << endl;
}

int main() {
    int a = 5;

    if (a < 10) {
        int a = 8;
        op2(a);
    } else {
        op1(a);
    }

    cout << a << endl;

    for (int i = 0; i < 2; i = i + 1) {
        if (i == 1) {
            int a = 20;
            op1(a);
            continue;
        }
        op2(a);
    }

    op1(a);

    return 0;
}
```

Can this program be compiled successfully? If so, please write the output after executing the program. If not, please identify the line numbers where the compilation errors occur and explain.

Answer:

Problem 5 [10 points] Array and C String

Draw the pattern that is printed by the following program.

You can use the grid provided for better alignment. Note that the grid space is more than enough.

You only need to draw the visible characters. For an empty space, e.g. `cout << " ";`, you can just leave the corresponding grid cell empty. The printing should start from the leftmost part as if this is a terminal. You should not leave empty spaces on the left unless the code prints white spaces.

```
#include <iostream>
using namespace std;

const char NULL_CHAR = '\0';

void func1(char s[], int m) {
    int n;
    for(n = 0; s[n] != NULL_CHAR; n++);

    int ind1 = m;
    int ind2 = n - 1;

    while (ind1 < ind2) {
        char temp = s[ind1];
        s[ind1] = s[ind2];
        s[ind2] = temp;
        ind1++;
        ind2--;
    }
}

void func2(char s[], char c1, char c2) {
    for(int i = 0; s[i] != NULL_CHAR; i++)
        if(s[i] == c1)
            s[i] = c2;
}

int main() {
    const int ROWS = 5;
    char grid[ROWS][10] = {
        "****----",
        "****----",
        "****----",
        "****----",
        "****----"
    };
```

```
    for (int i = 0; i < ROWS; i++) {
        func1(grid[i], i);
        func2(grid[i], '-', ' ');
        cout << grid[i] << endl;
    }

    return 0;
}
```

Answer:

Problem 6 [10 points] Recursion

Write a recursive function `alternating_sum`, which takes a positive integer `a` and returns the **alternating sum of its digits from right to left**.

More precisely:

- the rightmost digit is added,
- the next digit is subtracted,
- the next digit is added,
- and so on.

For example:

- `alternating_sum(7)` returns 7
- `alternating_sum(43)` returns $3 - 4 = -1$
- `alternating_sum(5728)` returns $8 - 2 + 7 - 5 = 8$
- `alternating_sum(120305)` returns $5 - 0 + 3 - 0 + 2 - 1 = 9$

You may assume that only positive integers will be used.

```
// Your implementation MUST BE RECURSIVE.
int alternating_sum(int a) {




}
```

Problem 7 [25 points] Function and Structure

In this question, you are developing a management system for a logistics company. The system tracks packages, their weights, and their shipping/delivery schedules. The data is organized using the following structures:

```
#include <iostream>
#include <cstring>
using namespace std;

const int MAX_STR_LEN = 20;
const int MAX_NUM = 5;

struct Date {
   int day; // Assume every month has 30 days
   int month;
   int year; // Assume every year has 360 days
};

struct Package {
   char id[MAX_STR_LEN]; // package id
   double weight; // weight of the package
   Date shipDate; // shipping date
   Date deliveryDate; // delivery date
};

struct LogisticsSystem {
   char name[MAX_STR_LEN];
   Package packages[MAX_NUM];
   int numPackages;
};
```

To complete the implementation, you will need to define the functions, namely, `calculateDays()`, `addPackage()`, `countOverduePackages()` and `getTotalWeightByDate()`, which are called in the `main()` function.

The main function also calls a function, `void printInfo(const LogisticsSystem& sys)`. You may assume that it has been defined and correctly implemented. It prints all the information stored in a `LogisticsSystem` object, `sys` and all the information stored in each of the `Package` elements in the `packages` array. It also calls `calculateDays()` to find and print the number of days between the shipping date and the delivery date.

```
// Definition of functions go here ...

int main() {
    LogisticsSystem mySystem = {"HKUST Express",
                                { {"PKG101", 10.5, {27, 3, 2026}, {30, 3, 2026}},
```

```
                                    {"PKG099", 20.0, {26, 3, 2025}, {1, 4, 2026}},
                                    {"PKG001", 18.3, {15, 2, 2026}, {20, 2, 2026}},
                                    {"PKG023", 2.3,  {20, 2, 2026}, {26, 3, 2026}}
                                  }, 4};

    Date date1 = {27, 3, 2026}, date2 = {3, 4, 2026}, date3 = {30, 3, 2026};
    addPackage(mySystem, "PKG102", 12.6, date1, date2);
    addPackage(mySystem, "PKG12", 5.2, date1, date3);
    printInfo(mySystem);

    cout << "Number of overdue packages (>5 days): "
         << countOverduePackages(mySystem, 5) << endl;

    Date checkDate = {27, 3, 2026};
    cout << "Total weight for 27-Mar: "
         << getTotalWeightByDate(mySystem, checkDate) << endl;

    return 0;
}
```

Here is the output of the program:

```
================================================
HKUST Express
------------------------------------------------
Number of packages: 5
ID      Weight  ShipDate  DeliveryDate  # of Days
------------------------------------------------
PKG101  10.5    27-3-2026 30-3-2026     3
PKG099  20      26-3-2025 1-4-2026      365
PKG001  18.3    15-2-2026 20-2-2026     5
PKG023  2.3     20-2-2026 26-3-2026     36
PKG102  12.6    27-3-2026 3-4-2026      6
================================================
Number of overdue packages (>5 days): 3
Total weight for 27-Mar: 23.1
```

Note:

- Your implementation should give the correct output as shown above.
- Your implementation should also be able to give correct results even if the values of the global constants, `MAX_STR_LEN` and `MAX_NUM`, are changed.
- Your implementation should also be able to handle different contents of `Date` objects, `Packages` objects and `LogisticsSystem` object.
- You can use the C string library functions given in the Appendix.

(a) [3 points] Implement the function `calculateDays()` to calculate the number of days between `startDate` and `endDate`.
E.g. if `startDate` is {30, 1, 2026}, `endDate` is {1, 3, 2026}, the function returns 31.
For simplification of calculations, you may assume the following:

- Every month has 30 days and every year has 360 days.
- `endDate` is always later than or equal to `startDate`.

```
int calculateDays(const Date& startDate, const Date& endDate)
{
    // Add your code here









}
```

(b) [7 points] Implement the function `countOverduePackages()` to count the number of packages in the system that took more than `maxDays` to be delivered.

```
int countOverduePackages(const LogisticsSystem& sys, int maxDays)
{
    // Add your code here









}
```

(c) [6 points] Implement the function `getTotalWeightByDate()` to find the total weight of packages with the same shipping date as `targetDate`.

```
double getTotalWeightByDate(const LogisticsSystem& sys, const Date& targetDate)
{
    // Add your code here









}
```

(d) [9 points] Implement the function `addPackage()` to add a package with the given `id`, `weight`, `shipDate` and `deliveryDate` at the end of the `packages` array of `sys` if the package `id` does not exist in the `packages` array. Do nothing if `id` exists in the `packages` array or if the `packages` array is full (i.e num of packages is already equal to `MAX_NUM`).

```
void addPackage(LogisticsSystem& sys, const char id[],
                double weight, const Date& shipDate, const Date& deliveryDate)
{
    // Add your code here







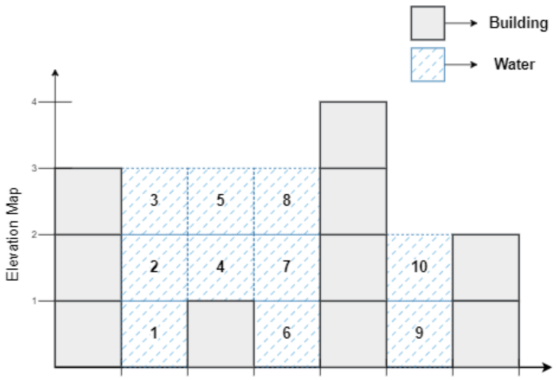


}
```

Problem 8 [26 points] Two-dimensional Array

In the city of Flatland, city planners represent the landscape as a 2D grid of size $h \times w$ (height $\times$ width). The city is composed of buildings represented by the character '#' and air represented by the character '.'.

If it rains heavily, due to gravity, some empty spaces ('.') will be filled with water, represented by '∼', if there are walls blocking both its left and right sides.



A structural leak occurs when a crack appears in a specific column index k. When this happens, any water connected horizontally to column k (including column k itself) will drain away until it hits a solid wall ('#').

Your task is to implement three **non-recursive** C++ functions that simulate the city's blueprint, the gravity-fed water accumulation, and the drainage effect of a structural leak.

```
// 1. Build the city skyline blueprint based on the 1D heights array
void buildCityMap(int heights[], int h, int w, char map[MAX_H][MAX_W]);
// 2. Fill the empty spaces with water based on gravity and walls
void applyGravityWater(char map[MAX_H][MAX_W], int h, int w);
// 3. Drain water connected to column k, stopping at wall obstacles
void applyStructuralLeak(char map[MAX_H][MAX_W], int h, int w, int k);
```

Note:

1. You may assume that all inputs (e.g., h, w, k) are valid
2. The top of the grid has the row index 0 and the bottom has the row index h-1
3. For simplicity, you may assume that both h and w are larger than 3
4. If the leftmost or rightmost column are NOT walls, they can't trap water. You may refer to test cases 3 and 4.

Read the skeleton program and sample outputs to better understand the tasks.

The skeleton program is given below. 4 sample testing cases are also provided.

```
#include <iostream>
using namespace std;

const int MAX_H = 10;  // Max height, h actual height
const int MAX_W = 10;  // Max width, w actual width

// --- UTILITY: Print with Row IDs and Column ID Footer ---
void printMap(char map[MAX_H][MAX_W], int h, int w) {
    for (int i = 0; i < h; i++) {
        cout << i << " "; // Print Row ID
        for (int j = 0; j < w; j++) cout << map[i][j] << " ";
        cout << endl;
    }
    // Print Column ID Footer
    cout << "  "; // Offset for Row ID column
    for (int j = 0; j < w; j++) cout << j % 10 << " ";
    cout << endl << "--------------------" << endl;
}

// --- UTILITY: Reset ---
void resetMap(char map[MAX_H][MAX_W], int h, int w) {
    for (int i = 0; i < h; i++) {
        for (int j = 0; j < w; j++) map[i][j] = '.';
    }
}

// --- Part 1: Blueprint ---
void buildCityMap(int heights[], int h, int w, char map[MAX_H][MAX_W]) {}
// --- Part 2: Gravity-Fed Filling ---
void applyGravityWater(char map[MAX_H][MAX_W], int h, int w) {}
// --- Part 3: Structural Leak ---
void applyStructuralLeak(char map[MAX_H][MAX_W], int h, int w, int k) {}

// --- Runner ---
void runTest(int heights[], int h, int w, int k, int testID) {
    char map[MAX_H][MAX_W];
    cout << "=== RUNNING TEST CASE " << testID << " ===" << endl;

    buildCityMap(heights, h, w, map);
    cout << "Initial Map:" << endl;
    printMap(map, h, w);

    applyGravityWater(map, h, w);
    cout << "After Gravity Filling:" << endl;
    printMap(map, h, w);

    cout << "Leak at column: " << k << endl;
```

```
    applyStructuralLeak(map, h, w, k);
    cout << "After Leak:" << endl;
    printMap(map, h, w);
}

int main() {
    int h1[] = {3, 0, 1, 0, 3};
    runTest(h1, 4, 5, 2, 1);

    int h2[] = {5, 0, 0, 0, 0, 0, 0, 0, 0, 5};
    runTest(h2, 6, 10, 5, 2);

    int h3[] = {0, 4, 0, 2, 0, 5, 0, 1, 0, 0};
    runTest(h3, 6, 10, 5, 3);

    int h4[] = {0, 4, 0, 2, 5, 5, 0, 1, 0, 0};
    runTest(h4, 6, 10, 4, 4);

    return 0;
}
```

The output of the sample testing cases.

```
=== RUNNING TEST CASE 1 ===
Initial Map:
0 . . . . .
1 # . . . #
2 # . . . #
3 # . # . #
  0 1 2 3 4
--------------------
After Gravity Filling:
0 . . . . .
1 # ~ ~ ~ #
2 # ~ ~ ~ #
3 # ~ # ~ #
  0 1 2 3 4
--------------------
Leak at column: 2
After Leak:
0 . . . . .
1 # . . . #
2 # . . . #
3 # . # . #
  0 1 2 3 4
--------------------
```

```
=== RUNNING TEST CASE 2 ===
Initial Map:
0 . . . . . .
1 # . . . . . #
2 # . . . . . #
3 # . . . . . #
4 # . . . . . #
5 # . . . . . #
  0 1 2 3 4 5 6 7 8 9
--------------------
After Gravity Filling:
0 . . . . . .
1 # ~ ~ ~ ~ ~ ~ #
2 # ~ ~ ~ ~ ~ ~ #
3 # ~ ~ ~ ~ ~ ~ #
4 # ~ ~ ~ ~ ~ ~ #
5 # ~ ~ ~ ~ ~ ~ #
  0 1 2 3 4 5 6 7 8 9
--------------------
Leak at column: 5
After Leak:
0 . . . . . .
1 # . . . . . #
2 # . . . . . #
3 # . . . . . #
4 # . . . . . #
5 # . . . . . #
  0 1 2 3 4 5 6 7 8 9
--------------------
```

```
=== RUNNING TEST CASE 3 ===
Initial Map:
0 . . . . . . . . . .
1 . . . . . # . . . .
2 . # . . . # . . . .
3 . # . . . # . . . .
4 . # . # . # . . . .
5 . # . # . # . # . .
  0 1 2 3 4 5 6 7 8 9
--------------------
After Gravity Filling:
0 . . . . . . . . . .
1 . . . . . # . . . .
2 . # ~ ~ ~ # . . . .
3 . # ~ ~ ~ # . . . .
4 . # ~ # ~ # . . . .
5 . # ~ # ~ # ~ # . .
  0 1 2 3 4 5 6 7 8 9
--------------------
Leak at column: 5
After Leak:
0 . . . . . . . . . .
1 . . . . . # . . . .
2 . # . . . # . . . .
3 . # . . . # . . . .
4 . # ~ # . # . . . .
5 . # ~ # . # . # . .
  0 1 2 3 4 5 6 7 8 9
--------------------
```

```
=== RUNNING TEST CASE 4 ===
Initial Map:
0 . . . . . . . . . .
1 . . . . # # . . . .
2 . # . . # # . . . .
3 . # . . # # . . . .
4 . # . # # # . . . .
5 . # . # # # . # . .
  0 1 2 3 4 5 6 7 8 9
--------------------
After Gravity Filling:
0 . . . . . . . . . .
1 . . . . # # . . . .
2 . # ~ ~ # # . . . .
3 . # ~ ~ # # . . . .
4 . # ~ # # # . . . .
5 . # ~ # # # ~ # . .
  0 1 2 3 4 5 6 7 8 9
--------------------
Leak at column: 4
After Leak:
0 . . . . . . . . . .
1 . . . . # # . . . .
2 . # . . # # . . . .
3 . # . . # # . . . .
4 . # ~ # # # . . . .
5 . # ~ # # # ~ # . .
  0 1 2 3 4 5 6 7 8 9
--------------------
```

(a) (4 points) Build the city skyline based on the 1D heights array

```
/* heights[]: Array where each element is the number of wall blocks at that index.
 * h: Total height of the city grid.
 * w: Total width of the city landscape.
 * map: The 2D grid to be filled, where '.' is air and '#' is a wall.
 */

void buildCityMap(int heights[], int h, int w, char map[MAX_H][MAX_W]){









}
```

(b) Fill empty spaces with water according to gravity and walls (non-recursive implementation)

(2 points) Briefly describe your program flow or algorithm in natural language in max 100 words.

(9 points) Implement with the C++ function.

```
/* map: The grid containing walls; fill valid basins with '~'.
 * h, w: height and width of the grid to process.
 */
void applyGravityWater(char map[MAX_H][MAX_W], int h, int w){

}
```

(c) Drain water connected to column k, stopping at wall obstacles (non-recursive implementation)

(2 points) Briefly describe your program flow or algorithm in natural language in max 100 words.

(9 points) Implement with the C++ function.

```
/* map: The grid containing water ('~') and walls ('#').
 * h, w: height and width of the grid.
 * k: The column index where the structural breach occurs.
 * replace water (~) with air (.) in the grid if it's drained
 */
void applyStructuralLeak(char map[MAX_H][MAX_W], int h, int w, int k){
// ask for additional page if the space is not enough

}
```

---------- END OF PAPER ----------

Appendix: Some Library Functions You May Need for Your Answers

```
/* strcpy() copies the string s2 to s1.
 * The string s1 should have enough memory to copy the characters
 * from the string s2.
 */
char* strcpy(char s1[], const char s2[]);

/* The strcmp() function compares the two strings s1 and s2.
 * It returns an integer less than, equal to, or greater than zero if s1 is
 * found, respectively, to be less than, to match, or be greater than s2.
 */
int strcmp(const char s1[], const char s2[]);


/* strlen() calculates the length of the C string s, excluding the terminating
 * null character.
 */
int strlen(const char s[]);
```

/* Rough work — You may detach this page */

/* Rough work — You may detach this page */

/* Rough work — You may detach this page */